Report Title

Generative AI as an Assistant to Developers

Prepared by:

Nada Ahmed Hamed Anwar

:University ID

2501589

Course:

Internet Applications and Electronic Platforms

Academic Year:

2026-2027



المقدمة:-

يشهد قطاع هندسة البرمجيات وتطوير الأنظمة ثورة تقنية غير مسبوقة بفضل القفزات الهائلة في مجال الذكاء الاصطناعي التوليدي (Generative AI). لم يعد دور الذكاء الاصطناعي مقتصرًا على الأتمتة البسيطة أو تحليل البيانات التقليدية، بل تطور ليصبح "شريكاً ذكياً" أو مساعداً شخصياً للمطور AI (Coding Assistant). تساعد هذه التقنيات اليوم في كتابة الأكواد ، اكتشاف الأخطاء، وتسريع دورة حياة تطوير البرمجيات (SDLC)، مما يطرح تساؤلات جوهرية حول طبيعة دور المطور المستقبلية وكيفية إعادة صياغة بيئات العمل البرمجية.

عرض الموضوع

1. مفهوم الذكاء الاصطناعي التوليدي في البرمجة

يُقصد بالذكاء الاصطناعي التوليدي في سياق التطوير تلك النماذج اللغوية الكبيرة (LLMs) المدربة على كميات هائلة من الأكواد البرمجية ومستودعات المصادر المفتوحة (مثل GitHub). تمتلك هذه النماذج القدرة على فهم النصوص الطبيعية وتحويلها إلى شيفرات برمجية دقيقة بلغات برمجة مختلفة، فضلاً عن فهم الشيفرات الموجودة وشرحها أو إعادة هيكلتها.

2. مجالات استخدام الذكاء الاصطناعي التوليدي كمساعد للمطور

- **الإكمال التلقائي واقتراح الأكواد (Code Completion):** تقديم اقتراحات ذكية وفورية للأسطر البرمجية التالية بناءً على سياق المشروع، مما يوفر وقتاً طويلاً في الكتابة اليدوية.
- **اكتشاف الأخطاء وإصلاحها (Error & Debugging Resolution):** تحليل رسائل الخطأ (Errors) واقتراح التصحيحات الفورية، بالإضافة إلى فحص الأكواد بحثاً عن ثغرات أمنية محتملة.
- **التوثيق وإنشاء الشروحات (Documentation Generation):** كتابة توثيق شامل للواجهات البرمجية (APIs) والدوال المعقدة تلقائياً، مما يسهل عملية صيانة الكود وفهمه من قبل أعضاء الفريق الجدد.
- **التحويل بين لغات البرمجة (Code Translation):** تسهيل عملية نقل الأنظمة القديمة من لغة برمجة إلى أخرى (مثل التحويل من Python 2 إلى Python 3 أو من Java إلى Kotlin) بسرعة وكفاءة.
- **كتابة الاختبارات البرمجية (Unit Testing):** توليد حالات اختبار تلقائية لضمان جودة البرمجيات والتحقق من صحة عمل الدوال البرمجية.

3. الإيجابيات والمكتسبات للمطورين

- **رفع الإنتاجية والسرعة:** إنجاز المهام الروتينية والمتكررة في أوقات قياسية.
- **خفض حاز التعلم:** مساعدة المطورين المبتدئين على فهم التقنيات الجديدة واستخدام إطارات عمل (Frameworks) معقدة بسرعة أكبر.
- **التركيز على التفكير الابتكارى:** إعفاء المطور من كتابة الأكواد التقليدية والتركيز على التصميم المعماري وحل المشكلات الإبداعية.

4. التحديات والمخاطر المحتملة

- **الأمن السيبراني وخصوصية البيانات:** خطر تسريب الأكواد السرية للشركات عند إدخالها في نماذج عامة.
- **دقة الأكواد (Hallucinations): قد** تنتج النماذج أكواداً تبدو صحيحة ظاهرياً لكنها تحتوي على ثغرات أو أخطاء منطقية خفية.
- **الاعتماد المفرط:** قد يؤدي الاعتماد الكلي على المساعد الذكي إلى تراجع المهارات البرمجية الأساسية لدى المطورين الجدد.

الخاتمة

يُعد الذكاء الاصطناعي التوليدي ثورة حقيقية تعيد تشكيل مستقبل هندسة البرمجيات، فهو ليس بديلاً عن المطور البشري بل هو أداة تمكين قوية تضاعف من كفاءته وقدرته على الابتكار. ومع استمرار تطور هذه التقنيات، تظل الحاجة ملحة للتعامل بحذر مع التحديات الأمنية والأخلاقية المرتبطة بها، لضمان بناء أنظمة برمجية آمنة وعالية الجودة.

المراجع

1. GitHub, Inc. (2023). *Research: quantifying GitHub Copilot's impact on developer productivity and satisfaction*. GitHub Blog.
2. OpenAI (2024). *GPT-4 Technical Report and Software Engineering Capabilities*. OpenAI Research.
3. Ziegler, A., et al. (2022). *The Impact of AI Assistants on Software Development Workflow*. IEEE Transactions on Software Engineering.
4. Fowler, M. (2023). *AI-Assisted Programming: Opportunities and Challenges in Modern Software Architecture*. Addison-Wesley.